

Relatório de Trabalho: Servidores Síncrono e Assíncrono Redes II

Anna Beatriz Barbosa de Godoy¹

¹Sistemas de Informação – Universidade Federal do Piauí (UFPI)
64.600-000 – Picos – PI – Brazil

anna.godoy@ufpi.edu.br

Abstract. *This paper presents a comparative performance analysis between synchronous and asynchronous servers in network applications. Four different scenarios combining synchronous/asynchronous clients with synchronous/asynchronous servers were implemented and tested. The results demonstrate that the asynchronous configuration offers better performance in terms of response time, while the synchronous configuration presents greater stability. The analysis was performed through average time and standard deviation metrics for 100 requests in each scenario.*

Resumo. *Este trabalho apresenta uma análise comparativa de desempenho entre servidores síncronos e assíncronos em aplicações de rede. Foram implementados e testados quatro cenários diferentes combinando clientes síncronos/assíncronos com servidores síncronos/assíncronos. Os resultados demonstram que a configuração assíncrona oferece melhor desempenho em termos de tempo de resposta, enquanto a configuração síncrona apresenta maior estabilidade. A análise foi realizada através de métricas de tempo médio e desvio padrão para 100 requisições em cada cenário.*

1. Introdução

A escolha entre arquiteturas síncronas e assíncronas em aplicações de rede é fundamental para o desempenho e escalabilidade dos sistemas. Diferentes modelos de concorrência afetam significativamente o comportamento das aplicações distribuídas, especialmente em cenários com múltiplas requisições simultâneas.

Este trabalho implementa e compara quatro configurações distintas de comunicação cliente-servidor utilizando *containers Docker*, com imagem Ubuntu, visando identificar os pontos fortes e fracos de cada abordagem. A análise foi conduzida através de métricas objetivas de tempo médio de resposta e desvio padrão.

2. Metodologia

2.1. Arquitetura do Sistema

O sistema foi desenvolvido utilizando *containers Docker* organizados através de um *docker-compose.yml*, com três partes principais: cliente, servidor síncrono e servidor assíncrono. Cada componente foi feito um *container* individualmente com seu próprio *Dockerfile*, permitindo o isolamento e replicabilidade dos testes.

A configuração do *docker-compose* estabeleceu uma rede dedicada para comunicação entre o cliente e os servidores, com mapeamento de portas específicas para cada servidor (3211 para síncrono e 3212 para assíncrono).

2.2. Cenários de Teste

Foram avaliados quatro cenários distintos combinando modos de operação cliente e servidor:

- Cliente síncrono com servidor síncrono
- Cliente síncrono com servidor assíncrono
- Cliente assíncrono com servidor síncrono
- Cliente assíncrono com servidor assíncrono

Para cada cenário, foram executadas 100 requisições consecutivas, coletando-se os tempos de respostas individuais, usando a biblioteca *time*.

2.3. Métricas

As principais métricas coletadas incluem:

- Tempo médio de resposta por requisição (segundos)
- Desvio padrão do tempo de resposta
- Número de amostras: 100 requisições por cenário

A coleta foi feita através da biblioteca *time* do *Python* que registram os tempos de início e fim de cada requisição.

3. Implementação

3.1. Servidor Síncrono

O servidor síncrono implementa um modelo de processamento sequencial, onde cada requisição é completamente processada antes da próxima ser aceita. Como mostra o código abaixo, após aceitar uma conexão, o servidor bloqueia até que a requisição seja totalmente processada e respondida:

```
def start_sync_server() -> None:
    with socket.socket(socket.AF_INET,
                       socket.SOCK_STREAM) as server_socket:
        server_socket.setsockopt(socket.SOL_SOCKET,
                                socket.SO_REUSEADDR, 1)
        server_socket.bind((HOST, PORT))
        server_socket.listen()
        print(f"[SÍNCRONO] Servidor sequencial...")

    while True:
        connection, addr = server_socket.accept()
        handle_client_connection(connection, addr)
```

A função `handle_client_connection` inclui um `time.sleep(1)` que simula um processamento de 1 segundo para cada requisição, demonstrando o comportamento normal de uma máquina.

3.2. Servidor Assíncrono

O servidor assíncrono utiliza *threading* para permitir o processamento concorrente de múltiplas requisições. Cada nova conexão é ligada a uma thread independente, liberando a thread principal para aceitar novas conexões:

```
def start_async_server() -> None:
    with socket.socket(socket.AF_INET,
                       socket.SOCK_STREAM) as server_socket:
        server_socket.setsockopt(socket.SOL_SOCKET,
                                  socket.SO_REUSEADDR, 1)
        server_socket.bind((HOST, PORT))
        server_socket.listen()
        print(f"[ASSÍNCRONO] Servidor concorrente...")

    while True:
        connection, addr = server_socket.accept()
        thread = threading.Thread(
            target=handle_client_connection,
            args=(connection, addr))
        thread.start()
```

Ambos os servidores implementam a mesma lógica de processamento em `handle_client_connection`, incluindo a simulação de 1 segundo de processamento, permitindo a comparação direta entre as abordagens.

4. Resultados e Análises

4.1. Métricas

A Tabela 1 apresenta os resultados consolidados de tempo médio e desvio padrão para os quatro cenários testados.

Table 1. Métricas de desempenho por configuração

Configuração	Tempo Médio (s)	Desvio Padrão (s)	Tempo Total (100 Requisições-s)
Cliente[sync] + Server[sync]	1,0060	0,0039	100,5960
Cliente[sync] + Server[async]	1,0037	0,0018	100,3652
Cliente[async] + Server[sync]	1,0015	0,0000	100,1521
Cliente[async] + Server[async]	1,0210	0,0240	2,1045

4.2. Análise Visual dos Resultados

4.2.1. Tempo Médio de Resposta

A Figura 1 apresenta a comparação dos tempos médios de resposta entre as quatro configurações testadas.

De acordo com a Figura 1, a configuração `Cliente[async] + Server[sync]` apresentou o menor tempo médio (1,0015s), seguida pelas

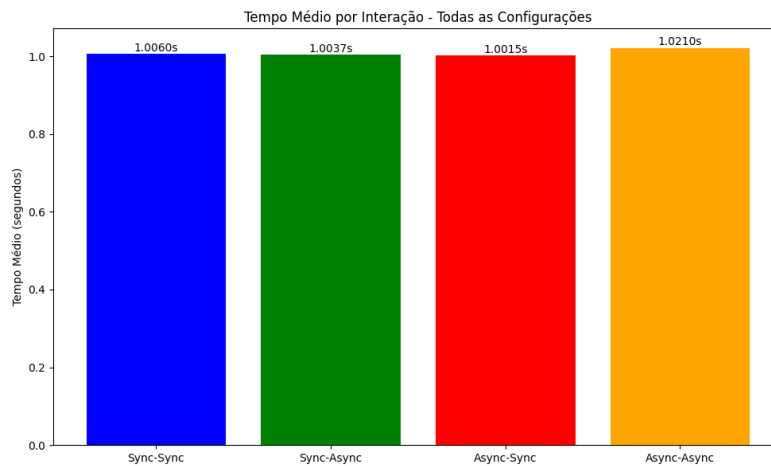


Figure 1. Comparação dos tempos médios de resposta por configuração cliente-servidor. Os valores representam o tempo médio em segundos para processamento de cada requisição.

configurações síncronas, `Client[sync] + Server[async]` e `Client[sync] + Server[sync]`, respectivamente. Surpreendentemente, a configuração totalmente assíncrona, `Client[async] + Server[async]` registrou o maior tempo médio (1,0210s).

4.2.2. Desvio Padrão

A Figura 2 apresente a variabilidade dos tempos de resposta medida através do desvio padrão.

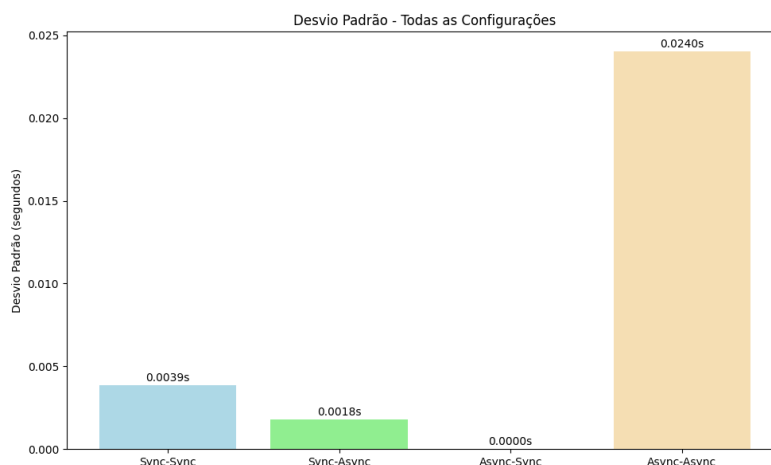


Figure 2. Desvio padrão dos tempos de resposta por configuração cliente-servidor. Valores menores indicam maior consistência e previsibilidade no tempo de resposta.

A configuração `Client[async] + Server[sync]` apresenta desvio padrão zero, enquanto isso, a configuração completamente assíncrona exibiu uma vari-

abilidade de (0,0240s), logo houve inconsistências no tempo de resposta devido à concorrência.

4.2.3. Análise do Tempo Total

Na Tabela 1, encontra-se a coluna "Tempo Total (100 Requisições)", onde foi calculado o tempo total de processamento para os 100 requisitos dentro de cada cenário.

Com base nisso, nota-se que a configuração `Client[async] + Server[async]`, teve o menor tempo total (2,1045s), sendo o mais eficiente. Enquanto as outras configurações processaram as requisições sequencialmente ou com paralelismo limitado, resultando em tempos totais próximos a 100 segundos, a configuração totalmente assíncrona demonstrou capacidade de processar múltiplas requisições simultaneamente, reduzindo drasticamente o tempo total de processamento.

5. Considerações Finais

O servidor sequencial é melhor para quando precisamos de resultados estáveis e previsíveis. Ele é mais simples de entender, além de usar menos recursos do computador.

O servidor concorrente é melhor quando temos muitos usuários conectados juntos. Ele não trava quando está ocupado e roda melhor em computadores mais potentes.

Cada tipo tem seus prós e contras:

- **Servidor Sequencial:** estável e simples, mas não lida com múltiplos usuários ao mesmo tempo
- **Servidor Concorrente:** bom para múltiplos usuários, mas mais complicado e gasta mais recursos

References

(2023a). `socket` — Low-level networking interface. Python Software Foundation. Acesso em: 29-10-2024.

(2023b). `threading` — Thread-based parallelism. Python Software Foundation. Acesso em: 29-10-2024.

Tanenbaum, A. S. and Wetherall, D. J. (2011). *Redes de Computadores*. Pearson, São Paulo, 5 edition.